A FREE PLAN FROM AGENTIC ACADEMY

CLAUDE CODE MEMORY PLAN

Hermes Agent's most-loved feature is memory. Here's the plan to build the same thing inside Claude Code, store, inject and recall on their own, with no server and no second subscription.

STORE INJECT RECALL

the idea, rebuilt for the tool you already have

START HERE

EVERY MEMORY SYSTEM ANSWERS 3 QUESTIONS

Before building anything, it helps to know what you're actually building. Store, inject and recall are the only three jobs a memory system has to do. Claude Code out of the box does all three badly: it saves almost nothing on its own, injects almost nothing at session start, and recall means grepping through old files by hand.

Store

When something matters, how does it get saved, and where?

Inject

When a session starts, what loads automatically, unasked?

Recall

When you ask about something old, how does it get found?

WHAT THIS PLAN BUILDS

01 Inject: the frozen snapshot
02 Store, part 1: agent-curated writes
03 Store, part 2: capture everything
04 Recall, part 1: search by meaning
05 Recall, part 2: cite the source
06 Bootstrap: don't start from zero

TAKEN FROM HERMES

01 INJECT: THE FROZEN SNAPSHOT

The facts Claude shouldn't have to dig for, current priorities, open questions, decisions still pending, should already be sitting in context before you type a word. Not searched for. Not summarized on the fly. Just there.

- A capped **working-memory file** with a handful of fixed sections: active threads, notes worth keeping, pending decisions.
- Keep the cap small on purpose. A memory file allowed to grow forever stops being memory and becomes a junk drawer.
- A separate **dated log file** for today, so daily detail doesn't threaten the cap.
- A **SessionStart hook** that reads both and injects them silently, no greeting, no recap.

WORTH KNOWING Treat the snapshot as frozen for the session. Writes take effect next session, not mid-conversation. That keeps the model's picture of "what's true right now" stable for the whole conversation.

TAKEN FROM HERMES

02 STORE, PART 1: AGENT-CURATED WRITES

You shouldn't be the one deciding what's worth remembering and manually editing a file to save it. Say "remember this" and the agent handles the rest, including not writing duplicate or contradictory entries.

- Build it as a **skill**, triggered by phrases like "remember," "note that," "forget about."
- Read the **entire** working-memory file first, every time, that's the dedup step.
- Decide the action: **add** a bullet, **replace** a superseded fact, or **remove** one on request.
- Check the cap. If adding would blow the budget, consolidate existing entries first.

WORTH KNOWING Write the actual judgment rules as editable instructions inside the skill, not hardcoded logic, so you can read them, disagree, and tune them as you learn what's worth keeping.

TAKEN FROM HERMES

03 STORE, PART 2: CAPTURE EVERYTHING

The curated file from Step 02 is a highlight reel: edited, capped. Separately, every single turn should get filed away durable and unedited, so months from now you can find the exact conversation a decision came from.

- A **Stop hook** extracts the last exchange from the session transcript.
- A cheap, fast model turns it into a short third-person summary.
- The summary is appended to today's log, under a clearly marked "auto-captured" section.
- The write is **idempotent**, hash the source turn, skip if it's already written.

WORTH KNOWING Run the hook detached, fire-and-forget, so it never blocks or slows down the session ending. Capture is a background chore, not something the user waits on.

HERMES DOESN'T HAVE THIS

04 RECALL, PART 1: SEARCH BY MEANING

Ask "what did we decide about payment processing" and if the actual conversation said "Stripe," a keyword search finds nothing. It's the single most common failure mode of memory systems, and exactly where most of them stop short.

- A **local vector database** in your project, plus a consistent **embedding model**. Re-embed everything if you switch models.
- A **chunking strategy** that splits by natural breaks (headings, sessions), not fixed windows.
- **Hybrid search**: vector and keyword search run in parallel, then merge.
- A cheap **rerank** by recency and source weight before returning results.

WORTH KNOWING You don't need anything more sophisticated than that to start. Add complexity later only if you actually hit its limits.

HERMES DOESN'T HAVE THIS

05 RECALL, PART 2: CITE THE SOURCE

When Claude recalls something, it should tell you the actual words, roughly when it was said, and where it came from, not a confident-sounding paraphrase. If nothing relevant turns up, it should say so instead of inventing an answer.

- Keep the **source file, date, and heading** attached to every stored chunk.
- Surface that metadata alongside the content on every recall, not just the text.
- A good answer reads: here's what was said, here's when, here's where it lives.

WORTH KNOWING This matters most exactly when the memory isn't your own: a teammate's decision, a client's stated preference. Vague recall is fine for jogging your own memory; it's a liability for anyone else's.

HERMES DOESN'T HAVE THIS

06 BOOTSTRAP: DON'T START FROM ZERO

The day you turn this system on, you already have months of Claude Code history sitting in your session logs, doing nothing. Don't leave it behind. Pull it in once, at install time.

- Walk your **existing session history** and extract meaningful turns, the same way the capture hook would.
- Summarize and write them into dated logs as if they'd been captured live.
- Index the result so it's **searchable immediately**.
- Guard it with a **sentinel file** so it only ever runs once.

WORTH KNOWING Do this and day one of your new memory system isn't empty. It already has everything you've built up to this point, searchable by meaning, cited by source.

BEFORE YOU CALL IT DONE

PROVE IT WORKS

Once it's wired up, confirm each of the three questions actually holds.

INJECT

Start a fresh session and ask "what were we working on?" without reminding it. It should already know, from the snapshot alone.

STORE

Say "remember that [some fact]," end the session, start a new one. The fact should be there, unprompted.

RECALL

Ask about something from weeks ago, using different words than you used originally. It should find it, and tell you where it came from.

> IF SOMETHING'S OFF
>
> If any of the three checks fail, that's the layer to debug, not the whole system. Inject, store and recall are independent; a broken one doesn't mean the others are broken too.

APPENDIX

PROMPTS YOU CAN PASTE STRAIGHT IN

Paste each of these into Claude Code, one at a time, in your project. Let it ask clarifying questions if it needs to; the prompt is a starting brief, not a rigid spec.

01 SNAPSHOT + INJECTION

Set up a memory/ folder in this project with a capped working-memory.md file (aim for a couple thousand characters max, with sections for active threads, notes worth keeping, and pending decisions) and a dated daily log file for today. Add a Claude Code SessionStart hook that reads both files and injects them as background context at the start of every session, silently, no greeting or recap. Treat the snapshot as frozen for the session, writes take effect next session.

02 CURATED WRITES

Create a skill that triggers on phrases like "remember this," "note that," "save," "forget about." When triggered, it should read the full working-memory file, check for duplicate or superseded entries, then add, replace, or remove content as appropriate, respecting the character cap from the snapshot file. Write the actual judgment rules as editable instructions inside the skill, not hardcoded logic.

03 FULL SESSION CAPTURE

Add a Claude Code Stop hook that extracts the last user/assistant exchange from the session transcript, summarizes it into a few third-person bullet points using a fast, cheap model, and appends it to today's dated log file under a clearly marked auto-captured section. Make the write idempotent by hashing the source turn. Run it detached so it never blocks session end. Also archive the raw transcript to a gitignored folder.

04 SEMANTIC SEARCH

Set up a local embedded vector database for this project. Write an indexing script that chunks the dated log files by natural section breaks, embeds each chunk, and stores it with metadata (source file, date, heading). Write a search script that runs vector and keyword search in parallel, merges the results, and reranks by recency and source weight.

05 CITATIONS

Update the search results so every returned chunk includes its source file, date, and heading alongside the content, so answers can cite exactly where a fact came from instead of paraphrasing without attribution.

06 HISTORICAL IMPORT

Write a one-time import script that walks my existing Claude Code session history, extracts meaningful turns the same way the Stop hook does, summarizes and writes them into the dated log format, then indexes them for search. Guard it with a sentinel file so it only runs once.

WANT IT RUNNING IN 10 MINUTES INSTEAD OF BUILDING IT?

Everything on these pages is already built, tested, and running inside Agentic OS. A one-line install, and your existing Claude Code history gets imported automatically: no hooks to write, no schema to design.

FIND IT ON SKOOL

SKOOL.COM/SCRAPES